

Grafana Alloy

This Helm chart deploys Grafana Alloy, a vendor-neutral distribution of the OpenTelemetry Collector.

Overview

Grafana Alloy is a flexible, performant, and vendor-neutral distribution of the OpenTelemetry Collector. Alloy offers native pipelines for OpenTelemetry, Prometheus, Pyroscope, Loki, and many other metrics, logs, traces, and profiling tools.

Features

- **Multi-Protocol Support:** Collect metrics, logs, traces, and profiles
- **Vendor Neutral:** Works with any observability backend
- **High Performance:** Efficient resource utilization and processing
- **Dynamic Configuration:** Hot-reload configuration without restarts
- **Service Discovery:** Automatic discovery of targets in Kubernetes
- **Flexible Pipelines:** Transform and route telemetry data

Prerequisites

- Kubernetes 1.19+
- Helm 3.0+
- Prometheus instance (for metrics forwarding)
- Loki instance (for logs forwarding, optional)

Installation

Using ArgoCD (Recommended)

This chart is automatically deployed via ArgoCD when included in the monitoring applications. The deployment is configured in the `bootstrap/argocd-apps-config` directory.

Note: The ArgoCD configuration uses the external Grafana Alloy chart directly from <https://grafana.github.io/helm-charts> with chart name `alloy` and version `0.4.0`.

Manual Installation

```
# Add Grafana Helm repository
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

# Install the chart
helm install grafana-alloy ./helm-charts/grafana-alloy -f values-
minikube-monitoring.yaml
```

Configuration

Environment-Specific Values

- **values.yaml**: Default configuration suitable for production environments with DaemonSet deployment
- **values-minikube-monitoring.yaml**: Development configuration with Deployment and reduced resources

Key Configuration Options

Deployment Type

```
alloy:
  controller:
    type: daemonset # or deployment
    replicas: 1      # only used for deployment type
```

Resource Configuration

```
alloy:
  resources:
    limits:
      cpu: 500m
      memory: 512Mi
    requests:
      cpu: 100m
      memory: 128Mi
```

Configuration Content

```
alloy:
  configMap:
    create: true
    content: |
      // Alloy configuration in River format
      logging {
        level  = "info"
        format = "logfmt"
      }

      // Add your pipeline configuration here
```

Usage

Accessing Alloy

```
# Port forward to access Alloy UI
kubectl port-forward -n grafana-alloy svc/alloy 12345:12345

# Access at http://localhost:12345
```

Configuration Format

Alloy uses the River configuration language. Here's a basic example:

```
// Prometheus metrics scraping
prometheus.scrape "kubernetes_pods" {
  targets = discovery.kubernetes.pods.targets
  forward_to = [prometheus.remote_write.default.receiver]
}

// Kubernetes service discovery
discovery.kubernetes "pods" {
  role = "pod"
}

// Remote write to Prometheus
prometheus.remote_write "default" {
  endpoint {
    url = "http://prometheus:9090/api/v1/write"
  }
}
```

Service Discovery

PROF

Alloy automatically discovers Kubernetes resources:

- **Pods:** Discovers pods with `prometheus.io/scrape: "true"` annotation
- **Services:** Discovers services with monitoring annotations
- **Endpoints:** Discovers service endpoints
- **Nodes:** Discovers cluster nodes

Metrics Collection

To enable metrics collection from your applications:

1. **Add annotations to your pods:**

```
metadata:
  annotations:
```

```
prometheus.io/scrape: "true"
prometheus.io/port: "8080"
prometheus.io/path: "/metrics"
```

2. **Expose metrics endpoint** in your application on the specified port and path

3. **Alloy will automatically discover and scrape** the metrics

Log Collection

For log collection (when enabled):

1. **Alloy mounts host paths** to access container logs
2. **Automatic discovery** of pod logs in `/var/log/pods`
3. **Forwarding to Loki** for centralized log storage

Monitoring

Alloy Metrics

Alloy exposes its own metrics on `/metrics` endpoint:

- Component health and status
- Pipeline throughput and latency
- Resource utilization
- Error rates and counts

Health Checks

- **Health:** `/-/healthy` on port 12345
- **Ready:** `/-/ready` on port 12345
- **Config:** `/-/config` on port 12345 (configuration view)

—
PROF

Security

RBAC

The chart creates appropriate RBAC resources:

- ServiceAccount for Alloy
- ClusterRole with permissions for service discovery
- ClusterRoleBinding to associate the role

Required permissions:

- `get`, `list`, `watch` on pods, services, endpoints, nodes
- `get` on configmaps and secrets (for configuration)

Security Context

Alloy runs with a restrictive security context:

```
securityContext:
  capabilities:
    drop:
      - ALL
  readOnlyRootFilesystem: true
  runAsNonRoot: true
  runAsUser: 473
```

Network Policies

When enabled, network policies restrict:

- Ingress traffic to necessary ports only
- Egress traffic to required destinations

Architecture

DaemonSet vs Deployment

DaemonSet (Production):

- Runs on every node
- Collects node-level metrics and logs
- Better for comprehensive monitoring
- Higher resource usage

Deployment (Development):

- Single replica
- Cluster-level monitoring only
- Lower resource usage
- Suitable for development environments

PROF

Data Flow

```
Applications → Alloy → Prometheus/Loki
    ↓           ↓
Annotations  Service Discovery
```

1. **Discovery:** Alloy discovers targets via Kubernetes API
2. **Collection:** Scrapes metrics/logs from discovered targets
3. **Processing:** Applies transformations and filtering
4. **Forwarding:** Sends data to configured backends

Troubleshooting

Common Issues

1. Service Discovery Issues

- Check RBAC permissions
- Verify Kubernetes API connectivity
- Review discovery configuration

2. Scraping Failures

- Verify target annotations
- Check network connectivity
- Review scrape configuration

3. Resource Issues

- Monitor CPU and memory usage
- Adjust resource limits
- Consider scaling strategy

4. Configuration Errors

- Check configuration syntax
- Review component logs
- Use configuration validation

Useful Commands

```
# Check Alloy pod status
kubectl get pods -n grafana-alloy

# View Alloy logs
kubectl logs -n grafana-alloy deployment/alloy

# Check discovered targets
kubectl port-forward -n grafana-alloy svc/alloy 12345:12345
# Visit http://localhost:12345/targets

# View current configuration
curl http://localhost:12345/-/config

# Check component health
curl http://localhost:12345/-/healthy

# Describe Alloy resources
kubectl describe deployment alloy -n grafana-alloy
```

Configuration Validation

```
# Validate configuration locally
alloy fmt config.river
alloy validate config.river

# Check configuration in cluster
kubectl exec -n grafana-alloy deployment/alloy -- alloy validate
/etc/alloy/config.river
```

Integration

With Prometheus

- Automatic service discovery
- Metrics forwarding via remote write
- Target relabeling and filtering
- Custom scrape configurations

With Loki

- Log collection from pods
- Structured log parsing
- Label extraction and enrichment
- Multi-tenant log forwarding

With Grafana

- Metrics visualization
- Log exploration
- Alerting integration
- Dashboard creation

—
PROF

Performance Tuning

Resource Optimization

```
alloy:
  resources:
    # Adjust based on cluster size and load
    limits:
      cpu: 1000m
      memory: 1Gi
    requests:
      cpu: 200m
      memory: 256Mi
```

Scrape Configuration

```
prometheus.scrape "optimized" {
  scrape_interval = "30s"
  scrape_timeout  = "10s"
  sample_limit    = 10000
  target_limit    = 1000
}
```

Queue Configuration

```
prometheus.remote_write "optimized" {
  endpoint {
    queue_config {
      capacity          = 10000
      max_shards        = 200
      max_samples_per_send = 5000
      batch_send_deadline = "5s"
    }
  }
}
```

Migration

From Prometheus Agent

1. **Export existing configuration**
2. **Convert to River format**
3. **Test configuration**
4. **Deploy Alloy**
5. **Verify data flow**
6. **Decommission old agent**

From Other Collectors

1. **Analyze current pipeline**
2. **Map components to Alloy**
3. **Create River configuration**
4. **Parallel deployment**
5. **Gradual migration**

Links

- [Grafana Alloy Documentation](#)
- [River Configuration Language](#)
- [Alloy Helm Chart](#)

- [Component Reference](#)